

Introduction

This document provides a detailed plan for manually testing the API endpoints for Swagger Petstore. The purpose is to ensure that all endpoints function as expected.

1. Add Pet

Endpoint URL	https://petstore.swagger.io/v2/pet
HTTP Method	HTTP method used POST
Description	Pet object that needs to be added to the store
Request Details	- Headers: -None- - Body (JSON):<pre>{ "id": 0, "category": { "id": 0, "name": "string" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre>
Expected Responses	- Code 200 with response body:<pre>{ "id": 9223372036854662000, "category": { "id": 0, "name": "string" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> - Code 400: Invalid/Unsupported Input

Test Cases:

1- TC_ADD_01

Title: Verify successful addition of a pet

Description: Test the POST /pet endpoint to ensure a new pet is added successfully.

Preconditions: API server is running.

Test Steps:

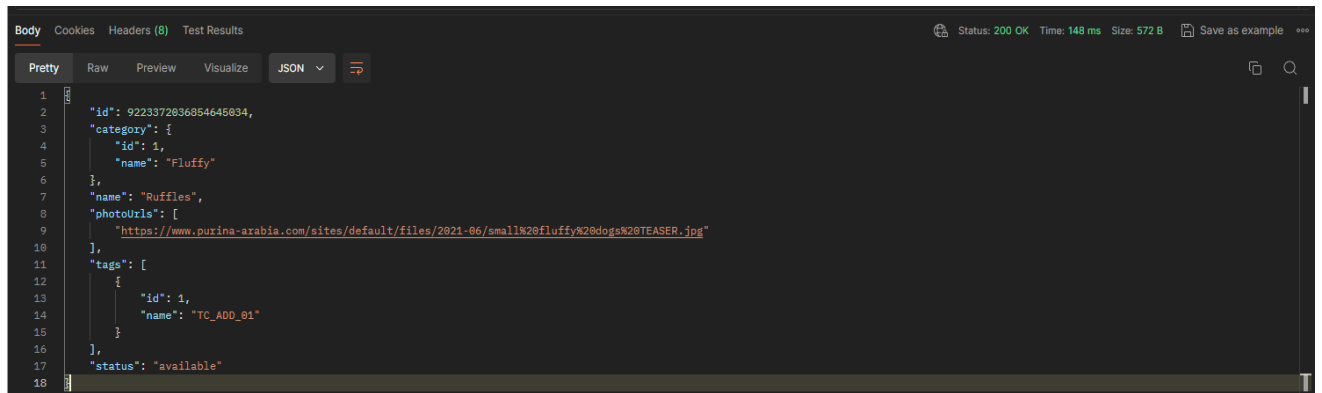
1. Open Postman.
2. Set HTTP method to POST.
3. Enter the endpoint URL.
4. Add request body.

```
{
  "id": 9223372036854645034,
  "category": {
    "id": 1,
    "name": "Fluffy"
  },
  "name": "Ruffles",
  "photoUrls": [
    "https://www.purina-arabia.com/sites/default/files/2021-06/small%20fluffy%20dogs%20TEASER.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "TC_ADD_01"
    }
  ],
  "status": "available"
}
```

5. Click Send.

Expected Results: Status Code 200, response body matches request body with an added ID.

Actual Results:



Pass/Fail: Pass

2- TC_ADD_02

Title: Verify addition of pet without body (Negative test case).

Description: Test the POST /pets endpoint with a missing name field to ensure appropriate error handling.

Preconditions: API server is running.

Test Steps:

- 1- Open Postman.
- 2- Set HTTP method to POST.
- 3- Enter the endpoint URL.
- 4- Select option 'none' in 'Body' section.
- 5- Click Send.

Expected Results: Status Code 400 for Invalid/Unsupported Input.

Actual Results:

POST https://petstore.swagger.io/v2/pet Send

Params Authorization Headers (7) Body Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (8) Test Results Status: 415 Unsupported Media Type Time: 583 ms Size: 2.53 KB Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "code": 415,
3   "type": "unknown",
4   "message": "com.sun.jersey.api.MessageException: A message body reader for Java class io.swagger.sample.model.Pet, and Java type class io.swagger.sample.model.Pet, and MIME media type application/octet-stream was not found.\nThe registered message body readers compatible with the MIME media type are:\n\n/>\n com.sun.jersey.core.impl.provider.entity.FormProvider\n com.sun.jersey.core.impl.provider.entity.StringProvider\n com.sun.jersey.core.impl.provider.entity.ByteArrayProvider\n com.sun.jersey.core.impl.provider.entity.FileProvider\n com.sun.jersey.core.impl.provider.entity.InputStreamProvider\n com.sun.jersey.core.impl.provider.entity.DataSourceProvider\n com.sun.jersey.core.impl.provider.entity.XMLJAXBElementProvider$General\n com.sun.jersey.core.impl.provider.entity.ReaderProvider\n com.sun.jersey.core.impl.provider.entity.DocumentProvider\n com.sun.jersey.core.impl.provider.entity.SourceProvider$StreamSourceReader\n com.sun.jersey.core.impl.provider.entity.SourceProvider$SAXSourceReader\n com.sun.jersey.core.impl.provider.entity.SourceProvider$DOMSourceReader\n com.sun.jersey.json.impl.provider.entity.JSONJAXBElementProvider$General\n com.sun.jersey.json.impl.provider.entity.JSONArrayProvider$General\n com.sun.jersey.json.impl.provider.entity.JSONObjectProvider$General\n com.fasterxml.jackson.jaxrs.json.JacksonJsonProvider\n com.sun.jersey.core.impl.provider.entity.XMLRootElementProvider$General\n com.sun.jersey.core.impl.provider.entity.XMLListElementProvider$General\n com.sun.jersey.core.impl.provider.entity.XMLRootElementProvider$General\n com.sun.jersey.core.impl.provider.entity.EntityHolderReader\n com.sun.jersey.json.impl.provider.entity.JSONRootElementProvider$General\n com.sun.jersey.json.impl.provider.entity.JSONListElementProvider$General\n com.sun.jersey.json.impl.provider.entity.JacksonProviderProxy\napplication/octet-stream ->\n com.sun.jersey.core.impl.provider.entity.ByteArrayProvider\n com.sun.jersey.core.impl.provider.entity.FileProvider\n com.sun.jersey.core.impl.provider.entity.InputStreamProvider\n com.sun.jersey.core.impl.provider.entity.DataSourceProvider\n com.sun.jersey.core.impl.provider.entity.RenderedImageProvider\n"
5 }
```

Pass/Fail: Pass

2. Get Pet by ID

Endpoint URL	https://petstore.swagger.io/v2/pet/{petId}
HTTP Method	HTTP method used GET
Description	Return single pet (if found).
Request Details	• Headers: -None- • Body (JSON): -None-
Expected Responses	• Code 200 with response body:<pre>{ "id": 9223372036854662000, "category": { "id": 0, "name": "string" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> • Code 404: Not Found

Test Cases:

3- TC_GET_01

Title: Retrieves details of a specific pet.

Description: Test the GET /pet/{petId} endpoint to ensure a new pet is added successfully.

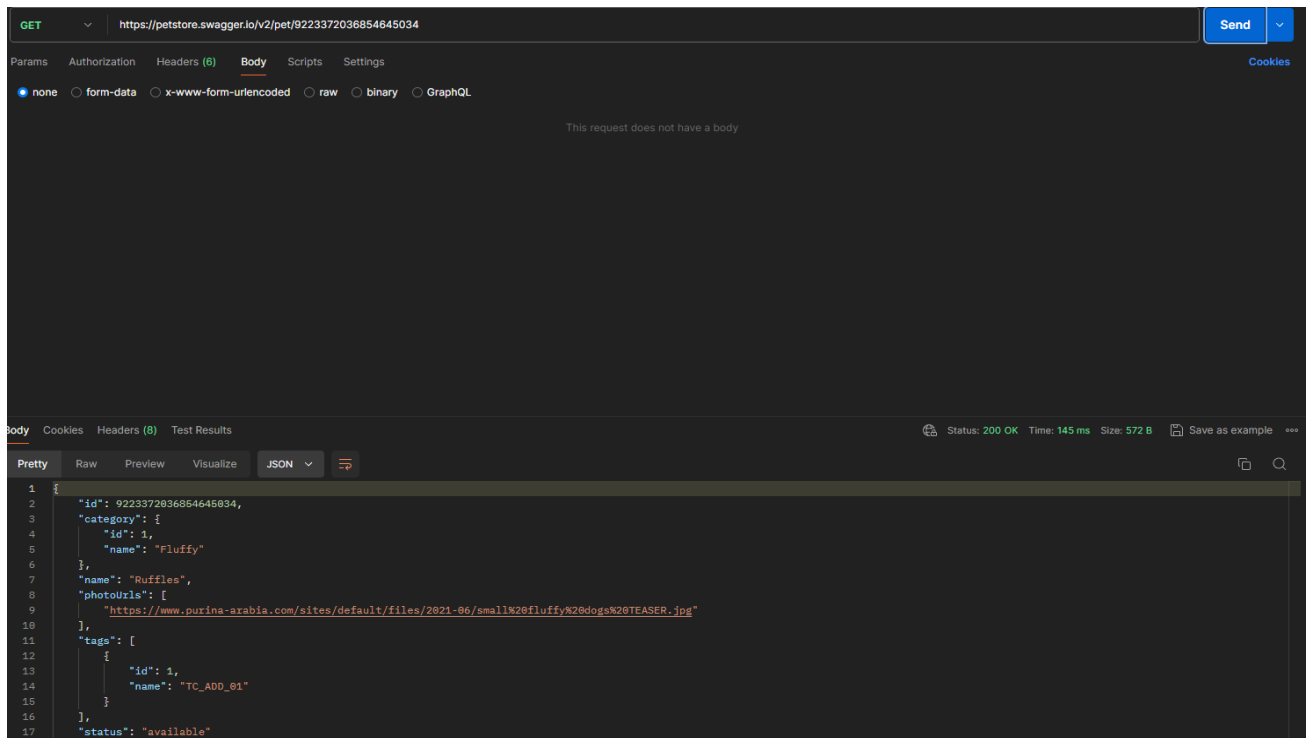
Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to GET.
3. Enter valid request URL with valid ID parameter.
4. Click Send.

Expected Results: Status Code 200, response body received with ID matches the ID sent as a parameter within the request URL.

Actual Results:



Pass/Fail: Pass

4- TC_GET_02

Title: Verify retrieval with non-existent pet ID (Negative test case).

Description: Test the GET /pet/{id} endpoint with a non-existent pet ID.

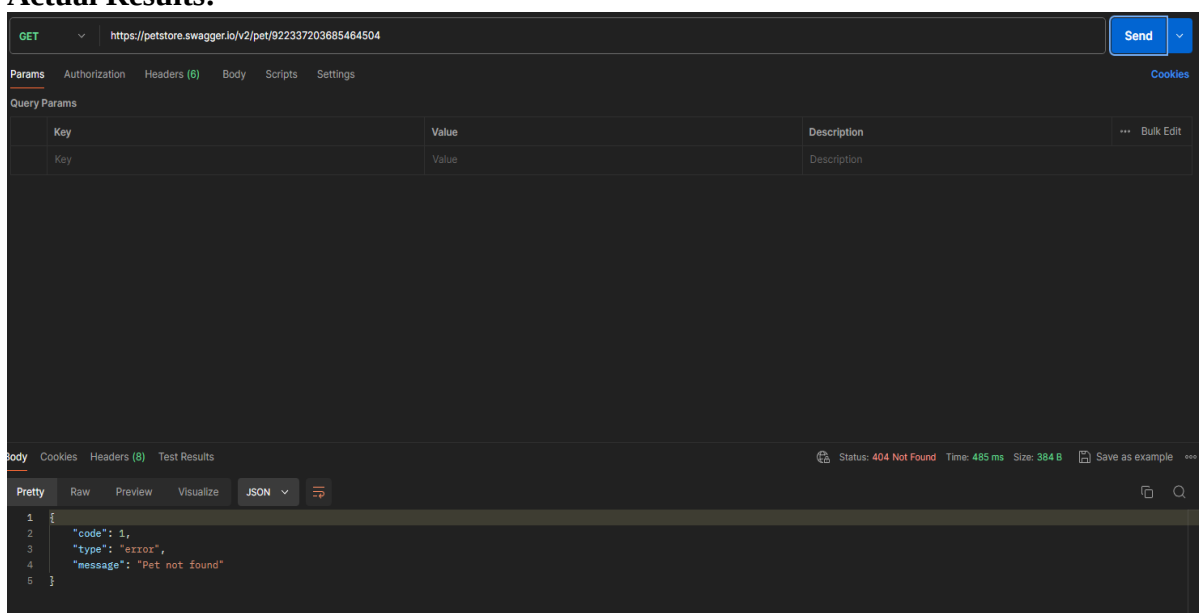
Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to GET.
3. Enter request URL with invalid/non-existent ID.
4. Click Send.

Expected Results: Status Code 404 not found Pet with given ID.

Actual Results:



Pass/Fail: Pass

3. Edit Pet data

Endpoint URL	https://petstore.swagger.io/v2/pet
HTTP Method	HTTP method used PUT
Description	Pet object that needs to be modified to the store.
Request Details	- Headers: -None- - Body (JSON):<pre>{ "id": 0, "category": { "id": 0, "name": "string" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre>
Expected Responses	- Code 200 with response body:<pre>{ "id": 9223372036854662000, "category": { "id": 0, "name": "string" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> - Code 404: Not Found

Test Cases:

5- TC_EDIT_01

Title: Edit Pet data for a given ID within the body

Description: Test the PUT /pet endpoint to ensure already existing pet is edited successfully.

Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to PUT.
3. Enter the endpoint URL.
4. Add request body with all pet data. (Changing category and tags id to 3. Besides changing tags name to "TC_EDIT_01")

```
{
  "id": 9223372036854645034,
  "category": {
    "id": 3,
    "name": "Fluffy"
  },
  "name": "Ruffles",
  "photoUrls": [
    "https://www.purina-arabia.com/sites/default/files/2021-06/small%20fluffy%20dogs%20TEASER.jpg"
  ],
  "tags": [
    {
      "id": 3,
      "name": "TC_EDIT_01"
    }
  ],
  "status": "available"
}
```

5. Click Send.

Expected Results:

1. Status Code 200, response body matches request body with an added ID.
2. For a double check, use search by ID from TC_GET_01 to make sure data has been modified.

Actual Results:

The screenshot displays the Swagger UI interface for a REST API. The top section shows a PUT request to the endpoint `https://petstore.swagger.io/v2/pet`. The request body is a JSON object representing a pet, with fields for `id`, `category`, `name`, `photoUrls`, `tags`, and `status`. The `id` is set to `9223372036854645034`, `category` is `{ "id": 3, "name": "Fluffy" }`, `name` is `"Ruffles"`, `photoUrls` is an array containing `"https://www.purina-arabia.com/sites/default/files/2021-06/small%20fluffy%20dogs%20TEASER.jpg"`, `tags` is an array containing `{ "id": 3, "name": "TC_EDIT_01" }`, and `status` is `"available"`.

The bottom section shows the response to the PUT request. The status is `200 OK`, the time is `630 ms`, and the size is `573 B`. The response body is a JSON object that matches the request body, but with the `id` field updated to `9223372036854645034`. The `category` field is `{ "id": 3, "name": "Fluffy" }`, `name` is `"Ruffles"`, `photoUrls` is an array containing `"https://www.purina-arabia.com/sites/default/files/2021-06/small%20fluffy%20dogs%20TEASER.jpg"`, `tags` is an array containing `{ "id": 3, "name": "TC_EDIT_01" }`, and `status` is `"available"`.

Pass/Fail: Pass

6- TC_EDIT_02

Title: Verify modification with non-existent pet ID (Negative test case).

Description: Test the PUT /pets/{id} endpoint with a non-existent pet ID.

Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to PUT.
3. Enter the endpoint URL.
4. Add request body with all pet data but with non-existing ID. (Changing category and tags id to 3. Besides changing tags name to "TC_EDIT_02")

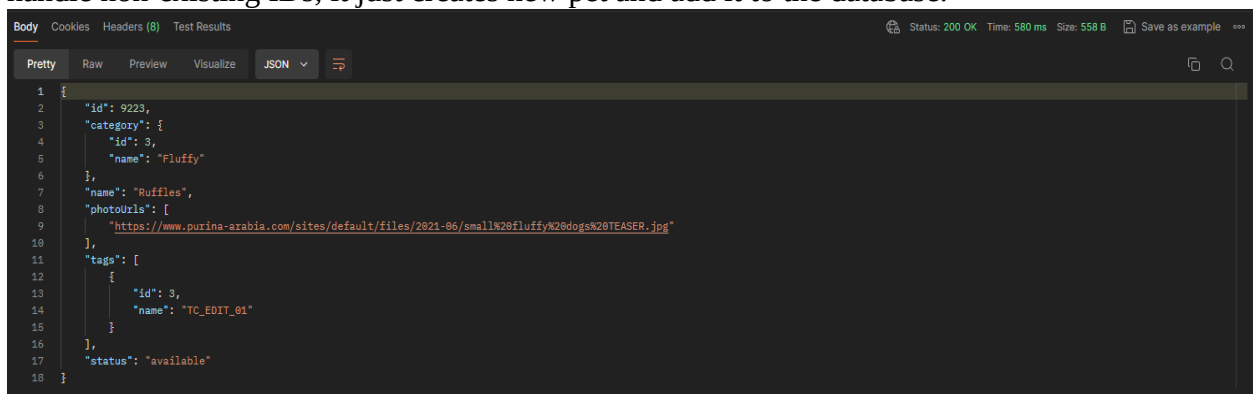
```
{
  "id": 9223,
  "category": {
    "id": 3,
    "name": "Fluffy"
  },
  "name": "Ruffles",
  "photoUrls": [
    "https://www.purina-arabia.com/sites/default/files/2021-06/small%20fluffy%20dogs%20TEASER.jpg"
  ],
  "tags": [
    {
      "id": 3,
      "name": "TC_EDIT_02"
    }
  ],
  "status": "available"
}
```

5. Click Send.

Expected Results: Status Code 404 not found Pet with given ID.

Actual Results:

The request appears to overwrite pre-existing data with given ID (if found). Otherwise, it doesn't handle non-existing IDs, it just creates new pet and add it to the database.



Pass/Fail: FAIL

4. Delete Pet

Endpoint URL	https://petstore.swagger.io/v2/pet/{petId}
HTTP Method	HTTP method used DELETE
Description	Delete Pet when ID matches.
Request Details	- Headers: -None- - Body (JSON): -None-
Expected Responses	- Code 200 with response body: <pre>{ "code": 200, "type": "unknown", "message": "9223372036854645034"}</pre> - Code 404: Not Found

Test Cases:

7- TC_DELETE_01

Title: Deletes details of a specific pet with its ID.

Description: Test the DELETE /pet/{petId} endpoint to ensure the pet is deleted successfully.

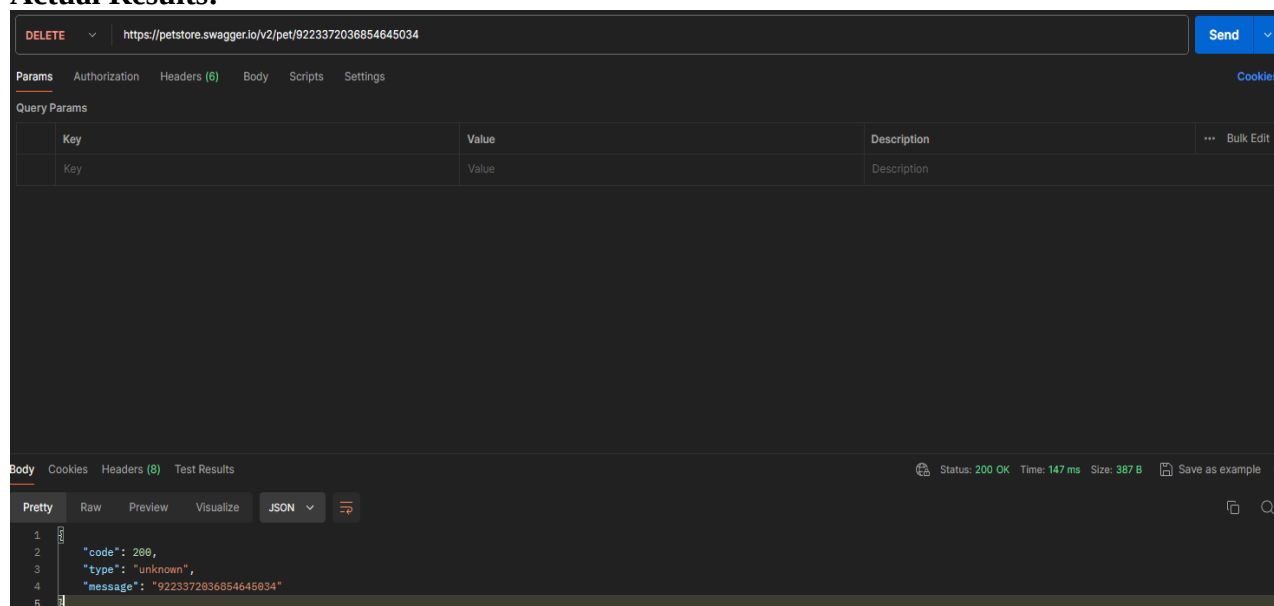
Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to DELETE.
3. Enter valid request URL with valid ID parameter.
4. Click Send.

Expected Results: Status Code 200, response body received with ID matches the ID sent as a parameter within the request URL.

Actual Results:



Pass/Fail: Pass

8- TC_GET_02

Title: Verify deleting the same pet ID for a second time (Negative test case).

Description: Test the DELETE /pet/{id} endpoint with a recently deleted pet ID.

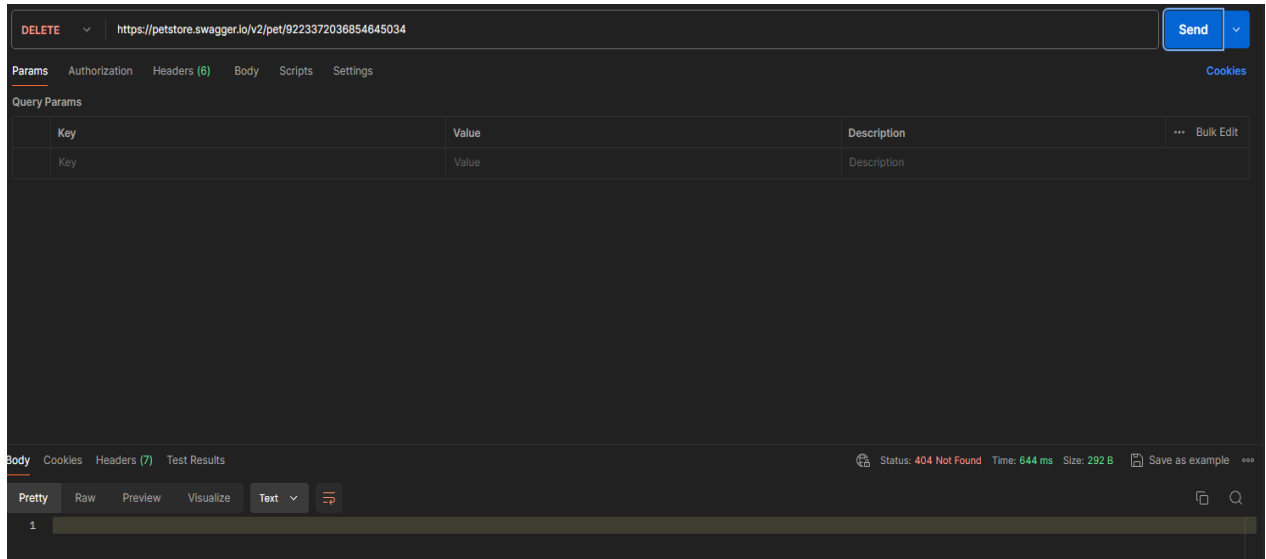
Preconditions: API server is running.

Test Steps:

1. Open Postman.
2. Set HTTP method to GET.
3. Enter valid request URL with valid ID parameter TWICE.
4. Click Send.

Expected Results: Status Code 404 not found Pet with given ID.

Actual Results:



Pass/Fail: Pass